Abjad at AISA-ArabicFC: A Diagnostic Study of Argument-Extraction Failures in Arabic Function Calling

Shahad Alsultan[1] **Randa Lakab**[2] **Abrar Almutiri**[3]

[1]Princess Nourah Bint Abdulrahman University, Saudi Arabia

[2]Ferhat Abbas Sétif 1 University, Sétif, Algeria

[3]Taibah University, Saudi Arabia

alsultanshahad38@gmail.com, lakabranda@gmail.com, Abraralshatry@gmail.com

Abstract

We describe Abjad's submission to the AISA-ArabicFC shared task (Track B), which targets Arabic function calling with explicit natural-language reasoning prior to the structured call. Rather than fine-tuning, our system builds a robust inference pipeline around the organizers' baseline model, AISA-AR-FunctionCall-Think, diagnosing and fixing two pipeline-level bugs that were silently depressing scores during development. On the shared task's 545-example development set, rescored with the organizers' own official evaluator (`normalize.py`/`eval_lib.py`), our submitted system reaches a function-name accuracy of 96.15%, an argument exact-match of 43.80%, and an official Track B score of 0.6825.

Beyond the original submission, we report a post-submission diagnostic finding: 117 of 500 positive predictions had a correct predicted function but a completely empty structured argument dictionary, and for 79 of these the model's own `<think>` span already stated every required gold value verbatim (an oracle upper bound, not an achieved result). Guided by this finding, we built a deterministic, schema-aware, gold-free repair that recovers such values only when they are unambiguously stated, together with a second, smaller schema-validity cleanup step; together they raise development-set ArgEM from 0.438 to 0.582 (+0.144) and Track B from 0.6825 to 0.7545 (+0.072), with zero observed regressions and no model weight updates, additional training data, or gold labels used at repair time. We present this as a diagnostic and pipeline-reliability contribution specific to our own submission: a substantial, measurable component of the argument-understanding gap observed in *our* development-set predictions from this baseline model is attributable to fixable output-serialization issues in our inference pipeline rather than semantic or reasoning failures, though genuine semantic errors remain and are characterized as the target for future modeling work. We do not claim this finding generalizes to other teams' pipelines or to the baseline model's behavior under different inference settings.

1 Introduction

Function calling, the ability of a language model to translate a natural-language request into a structured call to an external tool with correctly typed arguments, is an increasingly central capability for deploying language models in real applications. The AISA-ArabicFC shared task (Tuwaiq Academy AISA-ArabicFC Organizing Team, 2026) evaluates this capability specifically in Arabic, covering both Modern Standard Arabic and dialectal variation, using a baseline model that produces an explicit `<think>` reasoning span before emitting its function call.

This paper describes the system submitted by team Abjad: not a novel model or training procedure, but a carefully debugged inference pipeline built around the organizers' baseline model, together with a targeted error analysis. We identified and fixed two issues that silently corrupted our results during development: a stale checkpoint file that caused our generation loop to skip re-running samples, and an injected "anti-hallucination" reminder string that degraded the model's output format.

For this camera-ready version we additionally rescored our submitted, unmodified predictions using the organizers' *actual, released* scoring implementation (`normalize.py`/`eval_lib.py`) rather than the local reimplementation used at submission time, and conducted a further round of diagnostic analysis. We show that our original submission's low argument exact-match score is driven substantially by a pipeline-level structured-output failure — the model's reasoning trace routinely already contains the correct argument values, but the emit-

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1585–1591
October 28-29, 2026 ©2026 Association for Computational Linguistics

ted structured arguments are empty — rather than purely by argument-value misunderstanding, and we designed a deterministic, gold-free repair for this failure mode. Throughout this paper, we distinguish the *original submitted system* (rescored with the official evaluator) from *post-submission diagnostic experiments*: the latter are not a resubmitted or re-ranked competition entry.

Our contribution is therefore: (1) rescoring our submission with the actual official evaluator; (2) the discovery, via reproducible measurement rather than manual sampling, that a specific structured-output serialization failure accounts for a measurable share of the original argument-extraction gap; (3) a deterministic, gold-free repair pipeline that recovers a substantial portion of these serialization-related errors without any model weight changes; and (4) a grounded characterization of the errors that remain, which we argue are the genuine targets for future fine-tuning or decoding-level work.

The paper proceeds as follows: Section 2 covers task and dataset background; Section 3 describes our system and post-submission repair steps; Section 4 the experimental setup; Section 5 results and error analysis; Section 6 concludes.

2 Background

Task setup. Given a natural-language user request in Arabic, the system must produce a `<think>` reasoning span followed by either a structured function call or an explicit no-call decision. Two distinct correctness conditions are scored separately (Section 4): whether the predicted *tool name* matches gold, and whether the predicted *argument values* match gold under the organizers' normalization rules; the latter is not conditioned on the former.

Dataset and track. We use the `TuwaiqAcademy/AISA-ArabicFC` dataset released by the organizers: Arabic (MSA and dialectal) user utterances paired with gold tool calls from a fixed API-style function schema. The development split, on which all results here are computed, contains 545 examples (500 positive, 45 negative/`none`); we independently re-verified this split against the organizers' public Hugging Face release for this camera-ready version, confirming row counts and id alignment with our stored predictions. We participate in Track B, which additionally requires and scores the explicit `<think>` span (Section 4); we do not submit to Track A.

Related work and novelty. Tool use extends language models beyond text generation to structured, typed calls against APIs (Qu et al., 2024); most existing work is English-centric, and Ersoy et al. (2025) show that Arabic-specific tuning meaningfully affects tool-calling performance. Arabic function calling adds challenges beyond general Arabic NLP: dialectal variation, script normalization (digits, diacritics), and mapping Arabic-script mentions to standardized codes (ISO currency/language codes). AISA-ArabicFC (Tuwaiq Academy, 2026) is, to our knowledge, among the first shared tasks to combine these directly, using a baseline model, AISA-AR-FunctionCall-Think (Nacar et al., 2026), that couples function calling with explicit chain-of-thought. Unlike prior work reporting aggregate accuracy, our contribution is a diagnostic separating tool-selection from argument-normalization failures, plus pipeline-level lessons (Section 3) reusable regardless of fine-tuning.

3 System Overview

3.1 Base Model and Decoding

Our system uses the organizers' baseline model, `TuwaiqAcademy/AISA-AR-FunctionCall-Think`, loaded via the Hugging Face `transformers` library (Wolf et al., 2020). We run inference in float32 precision rather than float16, which we found necessary to avoid numerical instability (NaN outputs) during generation. We use greedy decoding (`do_sample=False`) with a repetition penalty of 1.1 and a generation budget of 300 new tokens, applied to the model's native chat template (`<start_of_turn>model`). A well-formed generation consists of a `<think>...</think>` reasoning span followed by either a structured call or an explicit indication that no tool applies.

3.2 Pipeline Debugging

We built our inference and evaluation pipeline from scratch on the provided train/dev splits. Two issues, visible only through direct inspection of raw model outputs rather than aggregate metrics, silently depressed our measured scores during development: (1) **stale checkpointing** — our generation loop resumed from a checkpoint file, but a stale checkpoint from an earlier, buggy run was silently treated as complete, causing later evaluations to score outdated predictions; and (2) a **prompt-injected anti-**

hallucination instruction — a reminder string telling the model not to invent unmentioned arguments interfered with its learned output format, reducing well-formed generations, and was removed after we compared raw outputs with and without it.

3.3 No Fine-Tuning

Our submitted system uses the baseline model's original weights, without parameter updates; we frame our contribution as diagnostic rather than leaderboard-optimized. Sections 3.5–3.6 describe post-submission, gold-free repairs for the structured-output failures identified in Section 5; genuine semantic errors are left as future work.

3.4 Output Parsing

We implemented a regex-based parser that extracts the `<think>` span and function call from raw output, typecasting argument values (number/string) per the tool's parameter schema before comparison against gold. This is the parser used to produce the originally submitted predictions.

3.5 Post-Submission Parser Repair

After submission, we investigated the composition of our system's argument exact-match failures using the organizers' officially released dev-set gold labels and scorer (Section 4). We found that 117 of the 500 positive-class predictions had a non-`none` predicted function but a completely empty structured `arguments` dictionary, despite the model's own `<think>` span routinely already stating the correct argument values in `key=value` form. This indicates a failure downstream of the model's reasoning — in how that reasoning was serialized into the structured call — rather than a reasoning failure itself.

We built a deterministic, schema-aware repair script that recovers arguments from `think` only when: (1) the predicted function is not `none`; (2) the structured `arguments` dictionary is empty; (3) the `think` span is non-empty; and (4) `think` contains an explicit, unambiguous `key=value`/`key: value` association for a key in the *predicted* function's own schema. Extraction uses only the predicted function name, the model's own `think` text, and tool schema metadata — **no gold labels, no hard-coded row identifiers**, and the predicted function name and `think` text are never altered. Ambiguous or non-schema-valid values are left unfilled rather than invented; the repair recovers only information the model had already stated explicitly.

3.6 Post-Submission Schema Cleanup

As a second, independent post-submission step, we inspected predicted argument *keys* against the predicted function's own declared schema and found a small number of predictions with keys that do not exist at all in that schema. We apply one general, deterministic, schema-derived (not gold-derived) rule: drop any predicted argument key not declared in the predicted function's own schema, regardless of whether the dropped key agrees or disagrees with gold.

We additionally apply a general safety guard: this cleanup is never applied if it would reduce a non-empty argument dictionary to completely empty, since gold arguments for scored examples are never empty, so emptying a prediction can only hurt or be neutral. This guard proved necessary because the official scorer's ArgEM check is independent of function-name correctness (Section 4): without it, a coincidental correct-argument match on a wrong-function prediction could be broken by cleanup that empties its argument dictionary.

4 Experimental Setup

Data and tools. We use the official train/development splits of `TuwaiqAcademy/AISA-ArabicFC`. All results here are on the 545-example development split (500 positive, 45 `none`); we lacked access to the blind test split.

Evaluator (revised). At submission time, we evaluated locally using our own reimplementation of the organizers' published scoring rules. For this camera-ready version, **all** dev-set numbers reported in this paper — including for the originally submitted, unmodified predictions — were recomputed using the organizers' actual released scoring implementation (`normalize.py`/`eval_lib.py`, obtained from the shared task's official Hugging Face Space). This changes some figures relative to the accepted submission (Section 5.1), since the official normalizer implements a broader set of closed-class aliases and a per-function `OPTIONAL_IGNORE` argument list that our earlier local reimplementation did not cover in full.

Metrics (corrected). Function-name accuracy (FnAcc) is computed over all development examples, including negative (`none`) ones. Argument exact-match (ArgEM) is computed only

over the 500 positive examples. We previously stated that ArgEM is forced to 0 whenever the predicted function name does not match gold; **this described our own earlier local reimplementation, not the organizers' actual evaluator**, whose `eval_lib.py` checks argument-dictionary equality independently of function-name correctness. Think-tag presence rate (ThinkRate) is the fraction of examples with a non-empty `<think>` span. The two official composite scores are unchanged: $\text{Overall}_A = 0.40\,FnAcc + 0.60\,ArgEM$
$Overall_B = 0.30\,FnAcc + 0.50\,ArgEM + 0.20\,ThinkRate$ with Overall_B the official ranking metric for Track B. All results in this paper, original and post-submission alike, are computed on the 545-row **development** split; the organizers' corrected baseline figures (Section 5.1) are reported on a separate **1,125-row blind test** split and are not numerically comparable to the development-set results here.

5 Results

5.1 Official Metrics

Table 1 summarizes results on the 545-row development split, recomputed with the organizers' official scorer.

The parser/serialization repair contributes $+0.132$ ArgEM and $+0.066$ Track B over the original submitted predictions; the schema-cleanup step contributes a further $+0.012$ ArgEM and $+0.006$ Track B; the total gain is $+0.144$ ArgEM and $+0.072$ Track B, with **zero regressions** at either stage — no previously argument-exact-match-correct example was broken, function-name predictions and `think` spans are identical across all three rows, and FnAcc/ThinkRate are unchanged throughout. The repaired-variant scores (0.7485, 0.7545) are post-submission development-set analysis figures, not a competition submission or leaderboard score. The organizers' corrected blind-test baselines (e.g., AISA-AR-FunctionCall-Think 270M: Track B 0.8074) are measured on the separate 1,125-row split described in Section 4 and are omitted from Table 1 accordingly.

5.2 Design-Decision Impact

Since we do not fine-tune the baseline model, we report no training-configuration ablations. The one pre-submission comparison available is the qualitative pipeline debugging in Section 3.2: inspection with and without the anti-hallucination reminder showed a clear drop in well-formed generations with it present, motivating its removal, though we did not re-quantify this with matched full-set scores (see Limitations).

5.3 Error Analysis

We replace our earlier manual-sample-based error characterization with measured, reproducible counts obtained by comparing predictions against the organizers' real development-set gold labels.

Diagnostic / oracle analysis. Of the 500 positive-class original submitted predictions, 117 had a non-`none` predicted function but a completely empty structured `arguments` dictionary. All 117 of these had the correct predicted function name and genuinely required arguments according to gold. Of these 117, 79 (67.5%) had every required gold argument value already present verbatim in the model's own `<think>` text, and 108 (92.3%) had at least one gold value present verbatim. **These 79 and 108 counts are oracle diagnostic observations** — they use gold labels purely to measure what could in principle be recovered from the existing `think` text, and they are *not* achieved corrections.

Actual gold-free implemented repair. The deterministic, schema-aware repair described in Section 3.5 — which uses no gold labels — modified the structured arguments of 114 predictions and made 66 additional positive examples argument-exact-match-correct, with zero regressions. The gap between the 79-row oracle upper bound and the 66 achieved corrections reflects the repair's deliberately conservative extraction criteria: it declines to fill a value whenever the `think` text's key/value association is not unambiguous under schema-restricted, literal pattern matching, in order to avoid inventing or guessing values.

ThinkRate composition. The official ThinkRate metric (0.8752) reflects 68 of the 545 development examples with a completely empty `<think>` span. Of these 68, 59 (86.8%) coincide with examples where the model itself predicted `none` (i.e., the model's own decision was that no tool call applies, and it did not additionally produce a reasoning trace for that decision); the remaining 9 (13.2%) are positive-class predictions — a non-`none` predicted function — with an empty `<think>` span

System	FnAcc	ArgEM	ThinkRate	Track A	Track B	ΔB
Original submitted predictions	0.9615	0.4380	0.8752	0.6474	0.6825	—
+ Parser/serialization repair	0.9615	0.5700	0.8752	0.7266	0.7485	+0.0660
+ Schema cleanup	0.9615	0.5820	0.8752	0.7338	0.7545	+0.0720 (total)

Table 1: Development-set results. All rows are computed on the *same* 545-example development split (500 positive, 45 negative) using the organizers' actual released `normalize.py`/`eval_lib.py`. The "Parser/serialization repair" and "Schema cleanup" rows are **post-submission camera-ready diagnostic experiments**, not part of the original competition submission and not a resubmitted or re-ranked leaderboard entry. ΔB is the change in Track B relative to the original submitted predictions.

despite a call being made. The shortfall from a perfect ThinkRate is therefore concentrated in `none`-decision cases rather than spread evenly across the dataset, and only a small minority (9/545, 1.65%) reflects missing reasoning alongside an actual function call.

Remaining errors after repair. After both post-submission repair steps, the dominant remaining failure groups are, in descending order of frequency: semantic/content value mismatches where the predicted and gold values differ even after the official normalizer's canonicalization; extra or missing schema-valid arguments, where the model over- or under-specifies arguments relative to gold despite both being valid for the predicted tool's schema; a smaller number of wrong function-name predictions; and a small number of fields — most notably `termination_type` — that the organizers' own published normalizer explicitly declines to canonicalize due to a large number of uncontrolled surface variants. We do not report exact post-final counts for these categories beyond what has been directly recomputed against the schema-cleanup predictions (Table 1).

Normalization finding. As a further post-submission diagnostic, we tested whether rewriting predicted argument *values* into their canonical surface form (e.g., mapping a spelled-out currency name to its ISO code) using the organizers' own published alias tables would improve scores beyond the repairs above. It did not: canonical value rewriting produced no additional ArgEM improvement, because the official scorer already applies the identical canonicalization function symmetrically to both the prediction and the gold answer at scoring time. This finding is specific to *additional pre-scoring canonical rewriting under this evaluator*; it does not imply that normalization in general is unhelpful, only that re-applying the same normalization the scorer already performs is redundant. The one genuine gain available from key-level (not value-level) cleanup — dropping schema-invalid argument keys — is already captured in Table 1.

6 Conclusion

We presented Abjad's submission to AISA-ArabicFC Track B together with a post-submission diagnostic analysis. Rescored with the organizers' official scorer, our submitted system reaches 96.15% function-name accuracy and 43.80% argument exact-match (Track B 0.6825). A substantial, measurable component of this gap is attributable to a pipeline-level structured-output failure rather than a failure of task comprehension: a deterministic, schema-aware, gold-free repair, together with a smaller schema-cleanup step, raises development-set ArgEM to 0.582 and Track B to 0.7545 (+0.144 ArgEM / +0.072 Track B), with zero regressions and no changes to model weights or training data. Canonical value rewriting alone yields no further gain, since the official scorer already applies the same canonicalization symmetrically. What remains — genuine value disagreements, function-selection errors, and organizer-acknowledged uncanonicalized fields — is increasingly model-level, and we identify constrained generation, prompt/decoding changes, or targeted fine-tuning on these residual error types as the appropriate next steps.

Limitations

Our originally submitted results used a local reimplementation of the organizers' published scoring rules rather than their released scorer; the camera-ready numbers in this paper (Table 1, all rows) instead use the organizers' actual `normalize.py`/`eval_lib.py`. All system results reported in this paper — for both the original submission and the post-submission repaired variants — are **development-set** results; final competition rankings are determined by the organizers on a

separate, held-out blind test split, to which we did not have access, and none of the numbers in this paper should be interpreted as revised leaderboard or test-set performance. The post-submission parser-repair and schema-cleanup variants (Sections 3.5–3.6, 5.1) were developed and evaluated entirely after our competition submission, using only the development split; we did not have blind test labels available to evaluate these repairs on held-out data, and we caution that iterative diagnostic development and evaluation on the same development split, however conservative, carries some risk of overestimating how well a fix would generalize relative to a genuinely held-out evaluation. The parser repair is fundamentally limited by what is recoverable from the model's own generated `think` text: it can only recover an argument value when the model's reasoning already states that value in an explicit, schema-matched, unambiguous form; it cannot correct cases where the model's reasoning itself is silent on or wrong about the required value. This work was conducted by a three-person team with limited compute. The stale-checkpoint and anti-hallucination-reminder fixes described in Section 3.2 were identified and validated during development via direct qualitative inspection of raw model outputs, comparing generations with and without each change on a small set of examples; this was sufficient to establish that each change was necessary to obtain well-formed output, but we did not preserve a matched, full-development-set quantitative comparison (with-bug vs. without-bug) under a single evaluator, so we cannot report a numerical before/after score for these two pre-submission fixes, unlike the post-submission repairs in Table 1, which are fully quantified.

Ethics Statement

This work uses the organizers' publicly released AISA-ArabicFC benchmark and baseline model as provided for the shared task; we did not collect, annotate, or release any new human-subject data. Our analysis is restricted to examining system outputs (model predictions and the released development-set inputs/gold labels) for the purpose of pipeline debugging and error characterization, and we do not make claims about the dataset's provenance, licensing terms, or content properties beyond what the organizers have documented. We report deterministic, rule-based post-processing methods only. No additional model training or human annotation was conducted for these experiments, and generative AI tools were not used to generate the reported experimental results.

Generative AI disclosure. Generative AI tools were used during the camera-ready preparation to assist with language editing, manuscript restructuring, and code review/debugging. All scientific claims, experimental procedures, reported results, and final manuscript content were reviewed and verified by the authors, who take full responsibility for the paper.

Acknowledgments

We thank the AISA-ArabicFC organizers (Tuwaiq Academy) for organizing the shared task and for publicly releasing the `TuwaiqAcademy/AISA-ArabicFC` benchmark dataset, the `AISA-AR-FunctionCall-Think` baseline model, and the official scoring and argument-normalization implementation (`normalize.py`/`eval_lib.py`), all of which this paper's camera-ready analysis relies on directly.

References

Asim Ersoy, Enes Altinisik, Husrev Taha Sencar, and Kareem Darwish. 2025. Tool calling for Arabic LLMs: Data strategies and instruction tuning. In *Proceedings of The Third Arabic Natural Language Processing Conference*.

Omer Nacar, Deema Alquffari, Saleh Alsharideh, Adeem AlOtaibi, Abdulaziz Alabdulkarim, Leen Alhazmi, Nada Alomar, Wareef Alzubaidi, Nada Alsultan, Ahmed Alrabghi, et al. 2026. From language to action in Arabic: Reliable structured tool calling via data-centric fine-tuning. *arXiv preprint arXiv:2603.16901*.

Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. 2024. Tool learning with large language models: A survey. *arXiv preprint arXiv:2405.17935*.

Tuwaiq Academy. 2026. Shared Task 15: AISA-ArabicFC: Arabic Function Calling for Agentic AI Systems. ArabicNLP 2026 Shared Task.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45.

A Normalization and Scoring Details

For replicability, we summarize the value-normalization rules applied before comparing predicted and gold argument values, based on the organizers' actual released `normalize.py`:

- **Digits & number words.** Arabic-Indic and Persian digits map to Western digits; common Arabic number words map to their digit equivalents.
- **Orthographic normalization.** Diacritics/tatweel are stripped; alef variants unify to bare alef; alef-maqsura and ta-marbuta are normalized; hamza seats are simplified; whitespace is collapsed.
- **Closed-vocabulary aliasing.** Countries, cities, currencies, languages, zakat asset types, and Quran search types map to canonical forms via lookup tables covering both Arabic and English/ISO surface forms. One field, `termination_type`, is explicitly *not* semantically aliased by the organizers due to a large number of uncontrolled surface variants, pending an organizer-approved canonical mapping.
- **Identifier and list fields.** Identifier-like fields (ID/IBAN/ passport/iqama/phone numbers, etc.) compare as normalized strings, preserving leading zeros; a small set of fields (`items`, `country`, `destination_country`) compare as unordered sets after splitting on common separators.
- **Optional/ignored arguments.** For a small, per-function set of arguments (`OPTIONAL_IGNORE` in the official implementation) that are declared in a tool's schema but never required by any gold answer, the argument is dropped from both prediction and gold before comparison.

Correction to the previously reported metric description. We previously stated that argument exact-match (ArgEM) for an example is "forced to 0 whenever the predicted function name does not match gold." This described our own local reimplementation used at submission time, not the organizers' actual evaluator. In the official `eval_lib.py`, ArgEM for a positive example is computed by directly comparing the (normalized) predicted and gold argument dictionaries, independently of whether the predicted function name is correct. We flag this correction for reproducibility; it does not materially change our headline findings, since function-name accuracy is uniformly high (96.15%) across all reported systems in this paper.